

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART IV KUBERNETES INTERNALS

API Server, etcd, Scheduler, Controllers, kubelet, Operators

Volume 4 of 16 Foundation Series

Prerequisites: Parts I, II, and III

Edition 2025-2026

TABLE OF CONTENTS

- 1. Kubernetes Architecture Overview 3
- 2. The API Server — Heart of the Control Plane 6
- 3. etcd — The Distributed State Store 12
- 4. The Scheduler — Placement Intelligence 17
- 5. Controller Manager — Reconciliation Engines 23
- 6. Cloud Controller Manager 28
- 7. The kubelet — Node Agent 31
- 8. kube-proxy — Service Networking 36
- 9. CoreDNS — Service Discovery 39
- 10. Admission Controllers 42
- 11. Custom Resource Definitions (CRDs) 47
- 12. The Operator Pattern 51
- 13. Leader Election and High Availability 56
- 14. API Request Lifecycle 60
- 15. Kubernetes HA Reference Architecture 64
- 16. Troubleshooting the Control Plane 67
- 17. Hands-On Exercises 70

CHAPTER 1

Kubernetes Architecture Overview

Kubernetes is a distributed system with a clear separation between the control plane (which manages cluster state) and the data plane (which runs workloads). Every architectural decision reflects two foundational principles: declarative desired-state management and level-triggered reconciliation. Understanding these principles is more valuable than memorising component names.

High-Level Architecture

```

CONTROL PLANE (typically 3 or 5 nodes for HA) +-----+ +-----+
+-----+ | API Server | | Scheduler | | Controller Mgr | | (kube-apiserver)| |
(kube-scheduler) | |(kube-ctrl-mgr) | +-----+ +-----+
+-----+ | | | +-----+ +-----+ +-----+ | |
+-----+ +-----+ +-----+ | etcd | | Cloud Controller | | (distributed KV) | |
Manager | +-----+ +-----+ DATA PLANE (N worker nodes)
+-----+ +-----+ +-----+ | Node | | +-----+
+-----+ +-----+ +-----+ | | | kubelet | | kube-proxy | | CoreDNS | | CNI/CSI | | |
+-----+ +-----+ +-----+ +-----+ | | +-----+ +-----+ +-----+
| | | Pod | | Pod | | Pod | | | +-----+ +-----+ +-----+ |
+-----+
  
```

Key Architectural Principles

- * **Declarative desired state:** Users declare WHAT they want (spec), not HOW to achieve it. The system continuously drives actual state toward desired state.
- * **Level-triggered reconciliation:** Controllers observe the current state of the world at all times, not just events. A controller that restarts can always recover by re-reading the current state. This is more robust than edge-triggered (event-based) systems.
- * **Optimistic concurrency:** All API updates include a resourceVersion. Concurrent modifications are detected by version mismatch and the caller must retry. No distributed locks required.
- * **Everything is an API object:** Every resource -- pods, nodes, namespaces, secrets -- is a typed API object stored in etcd and accessible via the API server. This uniformity enables a rich ecosystem of controllers, tools, and GitOps workflows.
- * **Extension without forking:** CRDs, Admission Webhooks, and the Operator pattern enable extending Kubernetes capabilities without modifying core code. The ecosystem's depth is built on these extension points.

CHAPTER 2

The API Server -- Heart of the Control Plane

The API Server (kube-apiserver) is the single entry point for all cluster operations. Every component -- scheduler, controllers, kubelet, kubectl -- interacts exclusively through the API server. It is the only component that reads from and writes to etcd. This centralisation is deliberate: it provides a single point for authentication, authorisation, admission control, and audit logging.

API Server Responsibilities

- * **REST API serving:** Serves the Kubernetes API (REST + gRPC + WebSockets) for all resource types: core (v1), apps, networking, storage, RBAC, and extension APIs.
- * **Authentication:** Validates the identity of every request using certificate-based auth, bearer tokens, OIDC, service account tokens, or webhook authentication.
- * **Authorisation:** Evaluates RBAC, ABAC, Node authoriser, or webhook authoriser policies to determine if the authenticated identity is permitted to perform the requested action.
- * **Admission control:** Mutating admission webhooks modify requests; validating admission webhooks approve or reject them. Built-in admission controllers enforce policy.
- * **Object validation:** Validates API objects against their schema before storage.
- * **etcd interface:** The only component permitted to read/write etcd. All others use the watch API to receive change notifications.
- * **Watch mechanism:** Clients subscribe to change streams. Controllers watch for changes to resources they manage without polling.
- * **Audit logging:** Records all API requests and responses for compliance and forensics.

API Request Processing Pipeline

```
Every API request traverses this pipeline in order: 1. TRANSPORT SECURITY TLS termination;
certificate validation 2. AUTHENTICATION Who is making this request? Tried in order: X.509
cert, bearer token, OIDC JWT, service account token, webhook, anonymous Result: username, UID,
groups, extra attributes 3. AUTHORISATION Is this identity allowed to perform this action on
this resource? RBAC: check Role/ClusterRole bindings Node authoriser: restrict kubelet to its
own node's resources Webhook: external policy engine (OPA, custom) Result: ALLOW or DENY (deny
by default) 4. ADMISSION CONTROL Mutating Webhooks (run first, can modify object) Object
Schema Validation Validating Webhooks (run after mutation, read-only) Built-in controllers:
NamespaceLifecycle, LimitRanger, ResourceQuota, ServiceAccount, PodSecurity, etc. Result:
object accepted (possibly modified) or rejected 5. STORAGE Object serialised, written to etcd
resourceVersion updated (monotonically increasing) 6. WATCH NOTIFICATION Subscribers
(controllers, scheduler, kubelet) notified of the change via their watch streams
```

API Groups and Versioning

Kubernetes APIs are organised into groups and versions:

```
Core group (legacy): /api/v1 Pod, Service, ConfigMap, Secret, PersistentVolume, Namespace,
Node Named groups: /apis/GROUP/VERSION apps/v1: Deployment, StatefulSet, DaemonSet, ReplicaSet
batch/v1: Job, CronJob networking.k8s.io/v1: Ingress, NetworkPolicy, IngressClass
```

```
storage.k8s.io/v1: StorageClass, PersistentVolumeClaim rbac.authorization.k8s.io/v1: Role,
RoleBinding, ClusterRole autoscaling/v2: HorizontalPodAutoscaler gateway.networking.k8s.io/v1:
Gateway, HTTPRoute API stability levels: v1: Stable GA -- no breaking changes without major
version bump v1beta1: Beta -- mostly stable, may change v1alpha1: Alpha -- may be removed
without notice # Discover all API resources: kubectl api-resources kubectl api-versions # Get
OpenAPI schema for a resource: kubectl explain pod.spec.containers.resources
```

Watch Mechanism -- How Controllers Stay in Sync

The watch mechanism is fundamental to Kubernetes' efficiency. Instead of polling the API server, every controller opens a long-lived HTTP connection (chunked transfer encoding) and receives a stream of events (ADDED, MODIFIED, DELETED) for resources it manages:

```
# Watch API: GET /api/v1/pods?watch=true&resourceVersion;=12345 # Server sends events as they
occur: HTTP/1.1 200 OK Transfer-Encoding: chunked {type: ADDED, object: {kind:Pod,
name:web-abc, ...}} {type: MODIFIED,object: {kind:Pod, name:web-abc, status:{phase:Running},
...}} {type: DELETED, object: {kind:Pod, name:web-abc, ...}} # Watch with label selector
(controller watches only its resources): GET
/api/v1/pods?watch=true&labelSelector;=app%3Dmyapp # Informer pattern (used by all
controllers): # 1. LIST (get all current objects + latest resourceVersion) # 2. WATCH from
that resourceVersion (get changes since list) # 3. If watch breaks: re-LIST and re-WATCH from
new resourceVersion # Local cache (Lister) serves reads without hitting API server # Only
watch events go to API server -> massive scale improvement
```

API Server Scalability and HA

The API server is stateless -- all state is in etcd. Multiple API server instances can run simultaneously behind a load balancer. Horizontal scaling is straightforward for read-heavy clusters. Write throughput is limited by etcd commit latency.

```
API server performance tuning: --max-requests-inflight=400 # max concurrent non-mutating
requests --max-mutating-requests-inflight=200 # max concurrent mutating requests
--request-timeout=60s # per-request timeout --watch-cache-sizes=pods#1000 # per-resource watch
cache size --etcd-compaction-interval=5m # how often etcd compacts history # Priority and
Fairness (APF) -- Kubernetes 1.20+ # Replaces --max-requests-inflight with flow-based rate
limiting # Prevents single noisy client from starving others kubectl get flowschemas kubectl
get prioritylevelconfigurations
```

CHAPTER 3

etcd -- The Distributed State Store

etcd is the only persistent state store in Kubernetes. Every cluster object -- every Pod, Deployment, Secret, ConfigMap, Service -- exists as a key-value pair in etcd. etcd's correctness guarantees underpin Kubernetes' consistency. Understanding etcd is essential for disaster recovery planning, performance tuning, and understanding why certain cluster operations behave the way they do.

etcd Architecture and Raft Consensus

etcd uses the Raft distributed consensus algorithm to provide strong consistency guarantees across a cluster of nodes. Raft ensures that all writes are agreed upon by a majority (quorum) of nodes before being committed:

```
Raft roles: Leader: Receives all writes; replicates to followers; heartbeats
Follower: Replicates log from leader; votes in elections
Candidate: Intermediate state during leader election
Write path: 1. Client sends write to leader
2. Leader appends to its log (uncommitted)
3. Leader sends AppendEntries RPC to all followers
4. Majority (N/2 + 1) of nodes acknowledge
5. Leader commits entry; applies to state machine
6. Leader notifies followers to commit
7. Response sent to client
Quorum requirements: 3-node cluster: needs 2 nodes (tolerates 1 failure)
5-node cluster: needs 3 nodes (tolerates 2 failures)
7-node cluster: needs 4 nodes (tolerates 3 failures)
RECOMMENDATION: Always run 3 or 5 etcd nodes in production. Never run an even number (split-brain risk without quorum benefit).
```

etcd Key-Value Structure for Kubernetes

Kubernetes stores all objects in etcd under a structured key hierarchy:

```
# etcd key structure: /registry/pods/default/my-pod /registry/pods/production/backend-xyz
/registry/deployments/apps/default/nginx-deployment
/registry/services/specs/default/kubernetes /registry/secrets/default/my-secret
/registry/configmaps/kube-system/kube-proxy /registry/namespaces/production
/registry/nodes/node-01 # Inspect etcd directly (use with extreme caution in production):
ETCDCTL_API=3 etcdctl \ --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \ --cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \ get /registry/pods/default/my-pod -w json | jq . #
Objects are stored as protobuf (binary) -- not human-readable JSON # Use kubectl get -o yaml
for human inspection # List all keys: etcdctl get / --prefix --keys-only | head -50
```

etcd Performance and Sizing

Metric	Target	Impact if Exceeded
fsync latency (p99)	< 10ms	Leader election instability, watch delays
Disk I/O bandwidth	200+ MB/s SSD	Write bottleneck, compaction lag
Database size	< 8GB (default quota)	etcd stops accepting writes
Number of objects	< 150,000 total	API server watch cache pressure

Metric	Target	Impact if Exceeded
Network RTT (inter-node)	< 2ms	Raft heartbeat timeouts, elections
CPU	Dedicated cores preferred	Scheduler interference causes latency spikes

etcd Backup and Disaster Recovery

etcd is the single source of truth for all cluster state. Its loss means complete cluster loss. Backup is non-negotiable:

```
# Create etcd snapshot: ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date
+Y%m%d-%H%M%S).db \ --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
--key=/etc/kubernetes/pki/etcd/healthcheck-client.key # Verify snapshot integrity:
ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-20250601-120000.db -w table # Restore from
snapshot (all etcd nodes must be stopped first): ETCDCTL_API=3 etcdctl snapshot restore
/backup/etcd-20250601-120000.db \ --data-dir=/var/lib/etcd-restored \ --name=etcd-0 \
--initial-cluster=etcd-0=https://192.168.1.10:2380 \
--initial-cluster-token=etcd-cluster-restore \
--initial-advertise-peer-urls=https://192.168.1.10:2380 # Backup frequency recommendation: #
Every 30 minutes for production clusters # Before every cluster upgrade # Store in separate
failure domain from cluster (S3, GCS, Azure Blob)
```

etcd Compaction and Defragmentation

etcd retains a history of all revisions (MVCC -- Multi-Version Concurrency Control). Without compaction, etcd database grows unboundedly. Compaction removes old revisions keeping only a configurable history window:

```
# etcd auto-compaction (configured in kube-apiserver): --etcd-compaction-interval=5m # compact
every 5 minutes # Manual compaction: REVISION=$(etcdctl endpoint status --write-out=json | jq
-r '[0].raftIndex') etcdctl compact $REVISION # Defragmentation (reclaims disk space after
compaction): # Schedule during maintenance window -- blocks etcd briefly etcdctl defrag
--endpoints=https://127.0.0.1:2379 # Monitor etcd database size: etcdctl endpoint status -w
table # Alert when db size > 6GB (approaching 8GB quota) # etcd metric:
etcd_mvcc_db_total_size_in_bytes
```

CHAPTER 4

The Scheduler -- Placement Intelligence

The Kubernetes scheduler (kube-scheduler) is responsible for selecting the optimal node for each unscheduled Pod. It is a watch-based controller that observes Pods with no spec.nodeName set and assigns them to nodes based on a sophisticated multi-factor scoring algorithm. Critically, the scheduler does not place Pods -- it only makes placement decisions. The kubelet on the selected node is responsible for actually running the Pod.

Scheduling Lifecycle

```
Complete scheduling lifecycle for a single Pod: 1. Pod created (kubectl apply / controller creates Pod) spec.nodeName is empty Pod enters Pending phase 2. Scheduler watch detects new unscheduled Pod Added to scheduling queue (sorted by priority) 3. FILTERING PHASE (feasibility) For each node, run all Filter plugins: NodeResourcesFit: node has enough CPU/memory NodeAffinity: node labels match nodeSelector/affinity TaintToleration: Pod tolerates node taints PodTopologySpread: spread constraints satisfied VolumeBinding: required volumes available on node NodeUnschedulable: node not cordoned PodAntiAffinity: anti-affinity rules not violated Result: list of feasible nodes (may be empty -> Pod stays Pending) 4. SCORING PHASE (optimisation) For each feasible node, run all Score plugins: LeastAllocated: prefer nodes with more free resources BalancedAllocation: prefer balanced CPU/memory usage NodeAffinity: higher score for preferred node affinity InterPodAffinity: prefer nodes with affinity pods ImageLocality: prefer nodes with image already pulled Result: ranked list of nodes with scores 5. NODE SELECTION Highest-scoring node selected Ties broken randomly (or by plugin) 6. BINDING Scheduler writes Pod.spec.nodeName = selected-node to API server API server stores binding in etcd 7. kubelet picks up bound Pod Starts container creation
```

Scheduling Framework -- Extension Points

The scheduling framework (introduced Kubernetes 1.15) provides structured extension points for customising scheduling behaviour without forking the scheduler:

Extension Point	Phase	Use Case
QueueSort	Queue management	Custom Pod priority ordering
PreFilter	Before filtering	Pre-compute state used in filtering
Filter	Feasibility	Node eligibility (node selectors, taints, resources)
PostFilter	After filtering	Handle unschedulable pods (preemption logic)
PreScore	Before scoring	Pre-compute state used in scoring
Score	Optimisation	Rank feasible nodes (affinity preference, balance)
NormalizeScore	After scoring	Normalise plugin scores to 0-100 range
Reserve	After selection	Reserve resources (volume binding, IP allocation)

Extension Point	Phase	Use Case
Permit	Hold/approve	Delay binding until conditions met (gang scheduling)
PreBind	Before binding	Pre-processing before API bind call
Bind	Binding	Write nodeName to API server (default: DefaultBinder)
PostBind	After binding	Cleanup, metrics, notifications

Scheduler Profiles and Multiple Schedulers

Kubernetes supports multiple scheduler profiles and custom schedulers. For AI workloads requiring GPU-aware scheduling, the NVIDIA GPU Feature Discovery plugin and custom schedulers (Volcano, YuniKorn) provide gang scheduling and GPU topology awareness:

```
# Use a custom scheduler for AI training jobs: apiVersion: v1 kind: Pod metadata: name:
gpu-training-job spec: schedulerName: volcano # Use Volcano scheduler for gang scheduling
containers: - name: trainer image: nvr.io/nvidia/pytorch:24.05-py3 resources: limits:
nvidia.com/gpu: 8 # Request 8 GPUs # Volcano gang scheduling -- all pods of a job must be
schedulable together: # Prevents partial allocation where 7/8 workers start but 1 is pending #
Critical for distributed training (all ranks must start simultaneously)
```

Node Affinity, Taints, and Tolerations

Three mechanisms control Pod-to-node placement. Understanding their interaction is critical for AI workload scheduling:

Mechanism	Direction	Hard/Soft	Primary Use Case
nodeSelector	Pod -> Node	Hard (required)	Simple label-based placement
Node Affinity (required)	Pod -> Node	Hard (required)	Complex label expressions
Node Affinity (preferred)	Pod -> Node	Soft (preferred)	Weighted placement hints
Pod Affinity	Pod -> Pod	Hard or Soft	Co-locate with specific pods
Pod Anti-Affinity	Pod -> Pod	Hard or Soft	Spread pods across nodes
Taints (NoSchedule)	Node -> Pod	Hard	Prevent scheduling (no toleration)
Taints (PreferNoSchedule)	Node -> Pod	Soft	Prefer not to schedule
Taints (NoExecute)	Node -> Pod	Hard + Eviction	Evict existing non-tolerating pods
Tolerations	Pod -> Taint	Enables	Allow scheduling on tainted nodes

Mechanism	Direction	Hard/Soft	Primary Use Case
TopologySpread	Pod -> Topology	Hard or Soft	Even distribution across zones/nodes

GPU Node Taint/Toleration Pattern

```
# Taint GPU nodes to prevent non-GPU workloads from consuming them: kubectl taint nodes
gpu-node-01 nvidia.com/gpu=true:NoSchedule # GPU workload Pod spec with toleration: spec:
tolerations: - key: nvidia.com/gpu operator: Exists effect: NoSchedule nodeSelector:
accelerator: nvidia-a100 # Only A100 nodes containers: - resources: limits: nvidia.com/gpu: 1
```

CHAPTER 5

Controller Manager -- Reconciliation Engines

The Controller Manager (kube-controller-manager) is a single binary that runs dozens of independent controllers, each responsible for reconciling one or more resource types. Each controller is a control loop that continuously watches for differences between desired state and actual state, and takes action to eliminate those differences.

The Reconciliation Loop Pattern

```
Every Kubernetes controller implements this pattern: func (c *Controller) reconcile(key string) error { // 1. Get desired state from cache (Lister -- no API call) desired, err := c.lister.Get(key) if err != nil { return err } // 2. Get actual state from system actual, err := c.getActualState(desired) // 3. Compute diff diff := computeDiff(desired, actual) // 4. Apply changes to drive actual -> desired return c.applyChanges(diff) } // The loop runs continuously: // - Triggered by watch events (ADDED, MODIFIED, DELETED) // - Re-queued on failure with exponential backoff // - Periodically re-synced (resync period) for safety // - Level-triggered: processes CURRENT state, not event deltas // This means: missed events don't cause divergence // Controller restart = re-read all state = fully recovered
```

Core Controllers and Their Functions

Controller	Watches	Manages	Reconciliation Action
Deployment	Deployments	ReplicaSets	Create/update/delete RS to match Deployment.spec
ReplicaSet	ReplicaSets, Pods	Pods	Create/delete Pods to match RS.spec.replicas
StatefulSet	StatefulSets	Pods, PVCs	Manage ordered Pod creation/deletion + PVC binding
DaemonSet	DaemonSets, Nodes	Pods	Ensure one Pod per node matching nodeSelector
Job	Jobs, Pods	Pods	Create Pods; track completions; retry failures
CronJob	CronJobs	Jobs	Create Jobs on schedule; manage history
HPA	HPAs, Pods, Metrics	Deployment/RS replicas	Scale replicas based on metrics
Node	Nodes	Node status, taints	Manage node lifecycle, evict pods from failed nodes
Namespace	Namespaces	Resources in namespace	Cascade delete on namespace deletion
ServiceAccount	ServiceAccounts	Secrets (token)	Create default service account token
EndpointSlice	Services, Pods	EndpointSlices	Track ready Pod IPs for each Service

Controller	Watches	Manages	Reconciliation Action
GarbageCollection	All resources	Owned resources	Delete resources when owner is deleted
TTLAfterFinished	Jobs	Jobs	Delete completed Jobs after TTL

Deployment Controller -- Rollout Strategy Deep Dive

The Deployment controller implements rolling updates through ReplicaSet management. Understanding this mechanism explains rollout behaviour and debugging approaches:

```
Rolling update internals: Before update: Deployment -> RS-v1 (replicas=3) Update triggered
(new image): 1. Deployment controller creates RS-v2 (replicas=0) 2. Scale RS-v2 up by 1
(maxSurge=1): RS-v1=3, RS-v2=1 (total=4, max allowed with maxSurge=1) 3. Wait for RS-v2 Pod to
become Ready 4. Scale RS-v1 down by 1 (maxUnavailable=0): RS-v1=2, RS-v2=1 5. Repeat until
RS-v2=3, RS-v1=0 6. RS-v1 retained (for rollback) but with 0 replicas # Key parameters:
strategy: type: RollingUpdate rollingUpdate: maxSurge: 1 # max pods above desired during
update maxUnavailable: 0 # max pods below desired during update # maxUnavailable: 0 =
zero-downtime rolling update # maxSurge: 0 = update in-place (one down, then one up) # Monitor
rollout: kubectl rollout status deployment/myapp kubectl rollout history deployment/myapp
kubectl rollout undo deployment/myapp # rolls back to previous RS
```

CHAPTER 6

Cloud Controller Manager

The Cloud Controller Manager (CCM) was introduced to separate cloud-provider-specific logic from the core Kubernetes codebase. It runs cloud-specific controllers that integrate Kubernetes resources with cloud provider APIs.

CCM Controllers

Controller	Cloud Resource	Kubernetes Trigger
Node controller	VM lifecycle (create/delete)	Node registers; cloud VM deleted -> Node condition
Route controller	VPC routing table entries	Pod CIDR assigned to node -> add route entry
Service controller	Cloud Load Balancer	Service type: LoadBalancer -> provision LB + assign IP
Volume controller	Cloud storage volumes	PersistentVolume creation/deletion via CSI

LoadBalancer Service -- CCM in Action

```
When you create a Service of type LoadBalancer, the CCM's service controller: 1. Detects new Service (type: LoadBalancer) via watch 2. Calls cloud provider API to provision a load balancer AWS: creates/configures ELB/NLB GCP: creates/configures Cloud Load Balancer Azure: creates/configures Azure Load Balancer 3. Configures health checks pointing to NodePort of the Service 4. Waits for LB to become active (IP allocated) 5. Updates Service.status.loadBalancer.ingress with the IP/hostname # Service spec: apiVersion: v1 kind: Service metadata: name: my-api annotations: service.beta.kubernetes.io/aws-load-balancer-type: external service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip spec: type: LoadBalancer selector: { app: my-api } ports: - port: 443 targetPort: 8443
```

CHAPTER 7

The kubelet -- Node Agent

The kubelet is the primary node agent. It runs on every worker node (and control plane nodes) and is responsible for: registering the node with the API server, watching for Pods assigned to its node, managing the container runtime to run those Pods, managing volumes, reporting node and Pod status, and executing health checks.

kubelet Responsibilities

```
kubelet responsibilities and the APIs it uses:
NODE MANAGEMENT: Register node with API server (POST /api/v1/nodes)
Report node capacity (CPU, memory, GPU, storage)
Update node conditions (Ready, MemoryPressure, DiskPressure, PIDPressure)
Report node heartbeat (every 10s via Lease object)
POD LIFECYCLE: Watch API server for Pods bound to this node
Call CRI to create Pod sandbox (network namespace via pause container)
Pull images via CRI ImageService
Create containers via CRI RuntimeService
Mount volumes (call CSI driver for persistent volumes)
Inject environment variables, ConfigMaps, Secrets
Execute init containers in order, then app containers
HEALTH MONITORING: Execute liveness probes (HTTP, TCP, exec, gRPC)
Execute readiness probes -> update EndpointSlices
Execute startup probes -> delay liveness until started
Restart containers on probe failure
STATUS REPORTING: Update Pod.status (phase, conditions, containerStatuses)
Report container exit codes and restart counts
Stream container logs to /var/log/pods/
```

kubelet Static Pods

Static Pods are managed directly by the kubelet, not the API server. The kubelet watches a directory on the node filesystem and creates Pods for any manifest files found there. This is how control plane components (API server, etcd, controller manager, scheduler) run on control plane nodes in kubeadm-bootstrapped clusters:

```
# Static Pod manifest directory (configured in kubelet config): ls /etc/kubernetes/manifests/
# kube-apiserver.yaml # kube-controller-manager.yaml # kube-scheduler.yaml # etcd.yaml #
kubelet watches this directory; changes take effect immediately # Static Pods appear in
kubectl get pods -n kube-system # but cannot be deleted via kubectl (kubelet recreates them) #
kubelet configuration file: /var/lib/kubelet/config.yaml # Key settings: staticPodPath:
/etc/kubernetes/manifests cgroupDriver: systemd # must match container runtime clusterDNS:
[10.96.0.10] # CoreDNS Service IP clusterDomain: cluster.local evictionHard: memory.available:
200Mi # Evict pods when node memory < 200Mi nodefs.available: 10% # Evict when node disk < 10%
nodefs.inodesFree: 5%
```

Pod Lifecycle Phases and Conditions

Phase	Meaning	Common Causes
Pending	Pod accepted but not yet running	Scheduler finding node; image pulling; volume binding
Running	At least one container running	Normal operation
Succeeded	All containers exited with code 0	Job/batch completion

Phase	Meaning	Common Causes
Failed	All containers terminated, at least one non-zero	Crash, OOM kill, exec failure
Unknown	Pod state cannot be determined	Node communication failure; node unreachable

CHAPTER 8

kube-proxy -- Service Networking

kube-proxy is a DaemonSet (or node daemon) that implements Kubernetes Service networking on each node. It watches Services and EndpointSlices and programs node-level network rules (iptables, IPVS, or eBPF) to route traffic to the correct backend Pods.

kube-proxy Modes

Mode	Implementation	Lookup Complexity	Max Scale	Recommendation
iptables	iptables DNAT chains	O(n) linear scan	~5,000 services	Legacy; avoid for large clusters
IPVS	kernel hash table	O(1)	100,000+ services	Use for medium clusters
eBPF (Cilium)	BPF maps + XDP	O(1)	Unlimited	Best for all cluster sizes
nftables (1.31+)	nftables rules	O(log n)	Better than iptables	Emerging; replaces iptables mode

Service Types and kube-proxy Behaviour

Service Type	ClusterIP	NodePort	External Access	kube-proxy Action
ClusterIP	Virtual IP (cluster-internal only)	None	None (internal only)	iptables DNAT: ClusterIP -> PodIP
NodePort	Virtual ClusterIP	Fixed port on all nodes	:	iptables DNAT: NodeIP:NodePort -> PodIP
LoadBalancer	Virtual ClusterIP	Auto NodePort	Cloud LB IP	CCM creates LB; iptables for NodePort
ExternalName	None (DNS CNAME)	None	External DNS name	CoreDNS CNAME; no iptables
Headless	None (DNS A records)	None	Direct Pod IPs	CoreDNS A records per Pod; no proxy

CHAPTER 9

CoreDNS -- Service Discovery

CoreDNS is the DNS server for Kubernetes clusters (replaced kube-dns in 1.11). It enables service discovery: pods can resolve Service names to ClusterIPs using DNS instead of hardcoding IP addresses. Every Pod has its `/etc/resolv.conf` configured to use CoreDNS as its DNS resolver.

DNS Resolution Hierarchy

```
# /etc/resolv.conf in every Pod: nameserver 10.96.0.10 # CoreDNS ClusterIP search
default.svc.cluster.local svc.cluster.local cluster.local options ndots:5 # DNS resolution
search order (ndots:5 means < 5 dots -> try search domains first): # Query: 'my-service' # 1.
my-service.default.svc.cluster.local -> Found (same namespace) # 2.
my-service.svc.cluster.local -> Not tried (found in step 1) # Query:
'my-service.other-namespace' # 1. my-service.other-namespace.default.svc.cluster.local -> Not
found # 2. my-service.other-namespace.svc.cluster.local -> Found # Query: 'api.external.com'
(6 labels, exceeds ndots:5) # 1. api.external.com -> Direct DNS query # DNS patterns: #
Service (same namespace): # Service (cross-namespace): . # FQDN: ..svc.cluster.local # Pod IP
(rare): 10-244-1-5.default.pod.cluster.local # StatefulSet Pod: ...svc.cluster.local
```

CoreDNS Corefile Configuration

```
# ConfigMap: kube-system/coredns (Corefile) .:53 { errors health { lameduck 5s } ready
kubernetes cluster.local in-addr.arpa ip6.arpa { pods insecure fallthrough in-addr.arpa
ip6.arpa ttl 30 } prometheus :9153 # Metrics endpoint forward . /etc/resolv.conf { # Forward
non-cluster queries to host DNS max_concurrent 1000 } cache 30 # Cache TTL for responses loop
# Detect forwarding loops reload # Auto-reload Corefile changes loadbalance # Round-robin DNS
load balancing } # Enterprise addition: forward internal domain to internal DNS
corp.internal:53 { forward . 10.0.0.53 10.0.0.54 cache 30 }
```

CHAPTER 10

Admission Controllers

Admission controllers are plug-ins that intercept API requests after authentication and authorisation but before persistence. They are the primary enforcement mechanism for policy, defaults, and security constraints in Kubernetes. Admission webhooks extend this mechanism to external policy engines.

Built-in Admission Controllers

Controller	Type	Function
NamespaceLifecycle	Validating	Reject operations on terminating namespaces
LimitRanger	Mutating+Validating	Apply default resource requests/limits from LimitRange
ResourceQuota	Validating	Enforce namespace resource quotas
ServiceAccount	Mutating	Auto-inject default ServiceAccount and token volume
PodSecurity	Validating	Enforce Pod Security Standards (Restricted/Baseline/Privileged)
NodeRestriction	Validating	Restrict kubelet to only modify its own Node and Pods
DefaultStorageClass	Mutating	Assign default StorageClass to PVCs without one
MutatingAdmissionWebhook	Mutating	Call external webhook to modify objects
ValidatingAdmissionWebhook	Validating	Call external webhook to approve/reject objects
Priority	Mutating	Assign PriorityClass to Pods
RuntimeClass	Mutating	Apply RuntimeClass overhead to Pod resource requests

Dynamic Admission Webhooks

Webhooks are the most powerful admission extension. A webhook is an HTTPS endpoint that receives an AdmissionReview request and returns an AdmissionReview response with approval/rejection or mutations:

```
# MutatingWebhookConfiguration example: apiVersion: admissionregistration.k8s.io/v1 kind:
MutatingWebhookConfiguration metadata: name: inject-sidecar webhooks: - name:
sidecar.injection.company.com clientConfig: service: name: sidecar-injector namespace:
istio-system path: /inject caBundle: rules: - apiGroups: [''] apiVersions: [v1] resources:
[pods] operations: [CREATE] namespaceSelector: matchLabels: istio-injection: enabled
failurePolicy: Fail # FAIL the request if webhook unreachable sideEffects: None # No
out-of-band side effects admissionReviewVersions: [v1] timeoutSeconds: 5 # Webhook must
respond within 5s
```

Webhook Failure Modes -- Critical Production Consideration

Webhook failurePolicy: Fail (the secure default) means that if your webhook is unavailable, NO PODS CAN BE SCHEDULED. A broken admission webhook has caused complete cluster outages. Always: (1) run webhooks with minimum 2 replicas, (2) set PodDisruptionBudget, (3) exclude kube-system namespace from webhook scope, (4) set realistic timeoutSeconds, (5) test webhook degradation.

OPA Gatekeeper and Kyverno

Feature	OPA Gatekeeper	Kyverno
Language	Rego (custom)	YAML/CEL (Kubernetes-native)
Learning curve	High (Rego)	Low (familiar YAML)
Policy types	ConstraintTemplate + Constraint	ClusterPolicy + Policy
Mutation support	Limited (assign/assignmetadata)	Full (strategic merge patch)
Generate resources	No	Yes (generate rules)
Image verification	Via external data	Native (verifyImages block)
Policy reports	Via audit	Native PolicyReport CRD
Maturity	CNCF Graduated	CNCF Incubating
Best for	Complex Rego policies, OPA ecosystem	K8s-native, ease of use, image signing

CHAPTER 11

Custom Resource Definitions (CRDs)

CRDs allow extending the Kubernetes API with custom resource types. Once a CRD is installed, users can create, read, update, and delete instances of the custom resource using `kubectl` and the Kubernetes API -- just like built-in resources. CRDs are the foundation of the Operator pattern and the entire CNCF ecosystem.

CRD Structure

```
apiVersion: apiextensions.k8s.io/v1 kind: CustomResourceDefinition metadata: name:
postgresclusters.postgres-operator.crunchydata.com spec: group:
postgres-operator.crunchydata.com names: kind: PostgresCluster plural: postgresclusters
singular: postgrescluster shortNames: [pgc] scope: Namespaced versions: - name: v1beta1
served: true storage: true schema: openAPIV3Schema: type: object properties: spec: type:
object properties: instances: type: array items: type: object properties: replicas: type:
integer minimum: 1 postgresVersion: type: integer enum: [14, 15, 16] additionalPrinterColumns:
- name: Age jsonPath: .metadata.creationTimestamp type: date subresources: status: {} # Enable
status subresource
```

CRD Best Practices

- * **Versioning strategy:** Plan API versions from day one. Use `v1alpha1` -> `v1beta1` -> `v1` progression. Implement conversion webhooks for multi-version support.
- * **Schema validation:** Define complete OpenAPI v3 schema. Reject invalid objects at admission time rather than discovering problems at reconciliation time.
- * **Status subresource:** Always enable status subresource. Allows status updates without triggering watch events on spec. Prevents accidental status overwrites by users.
- * **Printer columns:** Define `additionalPrinterColumns` for useful `kubectl` get output.
- * **Finalizers:** Use finalizers for resources requiring external cleanup (cloud resources, certificates, DNS records). Prevents deletion before cleanup completes.
- * **Owner references:** Set `ownerReferences` on resources created by the operator so that garbage collection automatically cleans up when the owner is deleted.

CHAPTER 12

The Operator Pattern

An Operator is a controller that manages a complex, stateful application (a database, message queue, ML model server) using the same reconciliation loop pattern as Kubernetes' built-in controllers -- but encoding application-specific operational knowledge. Operators automate the Day 2 operations that would otherwise require a human expert: provisioning, configuration, upgrades, scaling, backup, and recovery.

Operator Maturity Model

Level	Capabilities	Example
Level 1: Basic Install	Automated application install and configuration	Deploy MySQL with correct config
Level 2: Seamless Upgrades	Patch and minor version upgrades managed	Upgrade PostgreSQL 15 -> 16 safely
Level 3: Full Lifecycle	App lifecycle: backup, restore, failure recovery	Automated PITR restore for PostgreSQL
Level 4: Deep Insights	Metrics, alerts, log processing, workload analysis	Custom Grafana dashboards via operator
Level 5: Auto Pilot	Horizontal/vertical scaling, anomaly detection, tuning	Auto-tune PostgreSQL based on workload

Production Operators for AI Workloads

Operator	Manages	Key Capabilities
NVIDIA GPU Operator	NVIDIA driver, container toolkit, device plugin	Full GPU stack on Kubernetes nodes
KServe	ML model serving	Multi-framework inference, auto-scaling, canary
Kubeflow Operator	Kubeflow ML platform	Pipelines, notebooks, training jobs
Ray Operator	Ray distributed compute clusters	RayCluster, RayJob, RayService
Strimzi	Apache Kafka	Kafka cluster lifecycle, topic management, MirrorMaker
CloudNativePG	PostgreSQL clusters	HA Postgres, backup, replica management
Argo Workflows Operator	Workflow execution engine	DAG workflows for ML pipelines
Prometheus Operator	Prometheus monitoring stack	ServiceMonitor, PrometheusRule CRDs

Operator	Manages	Key Capabilities
cert-manager	TLS certificates	ACME, Vault, self-signed certs for all services
External Secrets Operator	Secret synchronisation	Sync from Vault, AWS SM, GCP SM

CHAPTER 13

Leader Election and High Availability

Kubernetes control plane components use leader election to ensure that only one instance of each controller is active at any time, preventing conflicting reconciliations. This enables running multiple replicas of the controller manager and scheduler for availability without causing split-brain.

Leader Election Mechanism

```
Kubernetes leader election uses the API server as a coordination primitive: 1. Each replica
tries to create/update a Lease object in kube-system: Name: kube-controller-manager (or
kube-scheduler) holderIdentity: leaseDurationSeconds: 15 renewTime: 2. The replica that
successfully holds the Lease is the leader 3. Leader renews Lease every leaseDuration/2
(~7.5s) 4. Non-leaders watch the Lease object: If Lease not renewed for leaseDurationSeconds
-> acquire Lease -> become leader 5. Old leader loses leadership and stops acting # View
current controller manager leader: kubectl get lease kube-controller-manager -n kube-system -o
yaml # Leader election parameters: --leader-elect=true --leader-elect-lease-duration=15s
--leader-elect-renew-deadline=10s --leader-elect-retry-period=2s
```

Kubernetes HA Reference Architecture

```
Production HA Kubernetes control plane: LOAD BALANCER (external, HA) api-lb.internal.corp:6443
Backends: control-plane-1:6443, control-plane-2:6443, control-plane-3:6443 CONTROL PLANE NODE
1 CONTROL PLANE NODE 2 CONTROL PLANE NODE 3 +-----+
+-----+ +-----+ +-----+ | kube-apiserver | | kube-apiserver |
| kube-apiserver | | kube-controller-manager | | kube-controller-manager | |
kube-controller-manager | | kube-scheduler | | kube-scheduler | | kube-scheduler | | etcd
(member 1) <---> | | etcd (member 2) <---> | | etcd (member 3) | +-----+
+-----+ +-----+ +-----+ Notes: - 3 API servers: all active,
behind LB (any can serve any request) - 3 Controller Managers: only LEADER is active; others
standby - 3 Schedulers: only LEADER is active; others standby - 3 etcd nodes: Raft quorum = 2;
tolerates 1 node failure - etcd MUST be co-located or within 2ms RTT of API server - For
5-node: tolerates 2 failures; recommended for critical production
```

CHAPTER 14

API Request Lifecycle

Tracing an API request from `kubectl` to running container reveals every component interaction. This complete lifecycle is essential knowledge for troubleshooting and understanding performance characteristics.

Complete Lifecycle: `kubectl apply` -> Pod Running

```
kubectl apply -f deployment.yaml PHASE 1: CLIENT-SIDE (kubectl) 1. kubectl reads deployment.yaml 2. kubectl discovers server API (GET /api, /apis -> API groups) 3. kubectl performs server-side apply: PATCH /apis/apps/v1/namespaces/default/deployments/myapp Content-Type: application/apply-patch+yaml 4. kubectl loads kubeconfig -> extracts credentials PHASE 2: API SERVER 5. TLS handshake (mutual TLS or bearer token) 6. Authentication: verify client certificate / OIDC token 7. Authorisation: RBAC check for PATCH deployments in default 8. Admission webhooks (mutating): inject sidecar, set defaults 9. Schema validation: verify Deployment spec is valid 10. Admission webhooks (validating): OPA/Kyverno policy check 11. Write Deployment to etcd 12. Return 200 OK to kubectl PHASE 3: CONTROLLER MANAGER 13. Deployment controller watch fires (ADDED/MODIFIED event) 14. Deployment controller reconciles: - Desired: replicas=3, image=myapp:v2 - Actual: RS-v1 exists (replicas=3, image=myapp:v1) - Action: create RS-v2 (replicas=0) 15. ReplicaSet controller watches RS-v2: - Desired: 1 Pod (maxSurge=1) - Actual: 0 Pods - Action: create Pod (spec.nodeName = empty) PHASE 4: SCHEDULER 16. Scheduler watch fires: new unscheduled Pod 17. Filter: find feasible nodes 18. Score: rank feasible nodes 19. Bind: PATCH Pod.spec.nodeName = worker-node-2 PHASE 5: KUBELET 20. kubelet on worker-node-2 watch fires: new Pod bound to its node 21. kubelet calls CRI: RunPodSandbox (create pause container + network ns) 22. CNI plugin called: configure veth pair, assign Pod IP 23. kubelet calls CRI: PullImage (if not cached) 24. kubelet calls CRI: CreateContainer 25. kubelet calls CRI: StartContainer 26. Init containers run in order (if any) 27. App containers start 28. kubelet starts probes (startup -> readiness -> liveness) PHASE 6: ENDPOINTS 29. EndpointSlice controller: Pod becomes Ready Add Pod IP to EndpointSlice for matching Services 30. kube-proxy: watch EndpointSlice update Update iptables/IPVS rules to include new Pod IP 31. Pod now receiving traffic via Service Total time: typically 5-30 seconds from kubectl apply to traffic-receiving Pod
```

CHAPTER 15

Troubleshooting the Control Plane

Issue: Pod stuck in Pending

Diagnosis: `kubectl describe pod` : look at Events section. Common causes: Insufficient resources: 0/3 nodes available; Insufficient cpu No matching nodes: node affinity/taint mismatch Volume binding: unbound PVC; no matching StorageClass Image pull failure: check `imagePullSecrets`, registry connectivity

Commands:

```
kubectl describe pod kubectl get events --sort-by=.lastTimestamp kubectl describe node (check Conditions, Capacity)
```

Issue: Pod stuck in CrashLoopBackOff

Diagnosis: Container starts then crashes repeatedly. Kubernetes uses exponential backoff.

Commands:

```
kubectl logs --previous (logs from last crash) kubectl describe pod (exit code, OOM kill) Exit code 137 = OOM killed (increase memory limit) Exit code 1 = application error (check app logs) Exit code 126/127 = entrypoint not found/executable
```

Issue: Service not routing traffic

Diagnosis: Pod Running but Service not reachable.

Commands:

```
kubectl get endpoints (check Pods are in endpoints) kubectl describe service (check selector matches Pod labels) kubectl exec -it -- curl : iptables-save | grep KUBE-SVC (verify iptables rules present) kubectl get pods -l app= (verify selector)
```

Issue: API server unreachable

Diagnosis: `kubectl` commands fail with connection refused or timeout.

Commands:

```
Check load balancer health systemctl status kube-apiserver (or: crictl ps | grep apiserver) journalctl -u kube-apiserver -n 100 etcdctl endpoint health Check certificates: kubeadm certs check-expiration
```

Issue: etcd alarm: database space exceeded

Diagnosis: All writes to cluster fail. etcd quota exceeded (default 8GB).

Commands:

```
etcdctl alarm list etcdctl compact $(etcdctl endpoint status -w json | jq '.[0].raftIndex') etcdctl defrag etcdctl alarm disarm Long-term: audit large objects (Events, large Secrets)
```

CHAPTER 16

Hands-On Exercises

Exercise 4.1 -- Watch the Reconciliation Loop

Observe the Deployment controller's reconciliation loop in real time:

```
# Terminal 1: Watch ReplicaSets and Pods kubectl get rs,pods -w # Terminal 2: Create and
update a Deployment kubectl create deployment demo --image=nginx:1.24 --replicas=3 # Watch
Terminal 1: RS created, 3 Pods created # Now update the image: kubectl set image
deployment/demo nginx=nginx:1.25 # Watch Terminal 1: new RS created, Pods scaled up/down
alternately # Observe rollout: kubectl rollout status deployment/demo # Check rollout history
(ReplicaSets retained): kubectl get rs kubectl rollout history deployment/demo # Roll back:
kubectl rollout undo deployment/demo # Old RS scales back up, new RS scales down
```

Exercise 4.2 -- Observe the Scheduling Decision

Create a Pod and observe the complete scheduling process:

```
# Watch scheduler events: kubectl get events --field-selector reason=Scheduled -w & # Create
an unschedulable Pod (request too much memory): kubectl run unmeet --image=nginx \ --overrides
='{ "spec": { "containers": [ { "name": "nginx", "image": "nginx", "resources": { "requests": { "memory": "99
99Gi" } } } ] } }' # Observe: FailedScheduling event kubectl describe pod unmeet | grep -A5 Events #
Clean up and create schedulable Pod: kubectl delete pod unmeet kubectl run meeting
--image=nginx --requests='cpu=100m,memory=128Mi' # Observe: Scheduled event with node name
kubectl get pod meeting -o jsonpath='{.spec.nodeName}'
```

Exercise 4.3 -- Inspect etcd State

Read raw Kubernetes state from etcd (read-only, safe to explore):

```
# Connect to control plane node and inspect etcd: # (assumes kubeadm-bootstrapped cluster)
ETCDCTL_API=3 ETCD_ARGS="--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \ --cert=/etc/kubernetes/pki/etcd/peer.crt \
--key=/etc/kubernetes/pki/etcd/peer.key" # List all Kubernetes keys: etcdctl $ETCD_ARGS get /
--prefix --keys-only | grep '/registry/' | head -30 # Count objects by type: etcdctl
$ETCD_ARGS get /registry/pods --prefix --keys-only | wc -l # Check etcd health and database
size: etcdctl $ETCD_ARGS endpoint status -w table # Watch etcd for changes (observe Kubernetes
in real time): etcdctl $ETCD_ARGS watch /registry/pods --prefix
```

End of Part IV -- Continue to Part V: Kubernetes Resources

Part V provides a comprehensive deep-dive into every core Kubernetes resource type: Pods, Deployments, StatefulSets, DaemonSets, Jobs, Services, Ingress, Gateway API, ConfigMaps, Secrets, PersistentVolumes, RBAC, NetworkPolicies, HPA, VPA, KEDA, PriorityClasses, PodDisruptionBudgets, and more. Each resource is examined with full YAML examples, internal implementation details, security implications, and production best practices.